

# Mathematical Modeling Tutorial

Operation Research

Dylan He

## 1 Graphic Theory

### 1.1 Basic Conceptions

Graph Theory focuses on graphs as its research object, which is an important part of discrete mathematics. Graph Theory is not only closely related to topology, computer data structures, and algorithms, but it is also becoming a key technology in machine learning.

In graph theory, the term “graph” does not refer to images or maps, but to a relational structure composed of vertices and edges connecting these vertices. Graphs provide a better way to deal with abstract concepts such as relationships and interactions, and they offer an intuitive visual way to think about these concepts. Here are some basic concepts about graphs:

- **Graph:** A graph is a relational structure consisting of a set of vertices and edges connecting these vertices.
- **Vertex (Node):** Points in a graph are called vertices, also known as nodes.
- **Edge:** The line connecting vertices is called an edge.
- **Parallel Edge:** Two edges with the same starting point and the same ending point are called parallel edges.
- **Cycle:** An edge where the starting point and the ending point coincide is called a cycle.
- **Digraph (Directed Graph):** Every edge in the graph has a direction, making it a directed graph.
- **Undirected Graph:** Every edge in the graph has no direction, making it an undirected graph.
- **Weighted Graph:** Each edge in the graph has one or more corresponding parameters, making it a weighted graph. The parameter associated with an edge is called its weight, which can represent distance, time, cost, etc., between two points.

- Degree: The number of edges connected to a vertex is called the degree of that vertex.

## 1.2 Introduction to NetworkX

NetworkX, abbreviated as NX, is a graph theory and complex network toolkit based on the Python language, used for creating, manipulating, and studying the structures, dynamics, and functions of complex networks.

NetworkX can describe graphs and networks in standard and non-standard data formats, generate graphs and networks, analyze network structures, build network models, design network algorithms, and visualize network graphs.

NetworkX provides classes, objects, graph generators, network generators, and plotting tools for graphs. It includes commonly used graph theory and network analysis algorithms, allowing for modeling, analysis, and simulation of graphs and networks.

The functionality of NetworkX is very powerful and complex, covering content that extends far beyond the scope of mathematical modeling. It is even challenging to provide a systematic overview due to its extensive nature. I will introduce the usage of NetworkX based on examples. You can read the official document for more information. <https://networkx.org/documentation/stable/>

## 1.3 Basic Operation

Let's get started with establishing a void graph. NX provides four kinds of graph classes.

- `nx.Graph()`: Void undirected graph
- `nx.DiGraph()`: Void digraph
- `nx.MultiGraph()`: Void multiple undirected graphs
- `nx.MultiDiGraph()`: Void multiple directed graphs

After successfully establishing a graph, we can operate on its nodes and edges. Every node of the graph has a unique property "label" expressed by integers or strings. We can also define new properties when we add those nodes to the graph.

```

1 import NetworkX as nx
2
3 G1 = nx.Graph()
4
5 G1.add_node(1) # add node 1 to G1
6 G1.add_node(1, name='n1', weight=1.0)
7 # Add node 1 and define its name and weight
8 G1.add_node(2, date='May-16') # Add node 2 and define its property: time
9 G1.add_nodes_from([3, 0, 6], dist=1)

```

```

10 # Add multiple nodes and define their properties dist
11 G1.add_nodes_from(range(10, 15)) # Add nodes 10-14 to G1
12
13 # View labels and properties of nodes
14 print(G1.nodes()) # View the list of nodes
15 # [1, 2, 3, 0, 6, 10, 11, 12, 13, 14]
16 print(G1._node) # View the dictionary of properties.
17 # {1: {'name': 'n1', 'weight': 1.0}, 2: {'date': 'May-16'}, 3: {'dist': 1}
18 # 0: {'dist': 1}, 6: {'dist': 1}, 10: {}, 11: {}, 12: {}, 13: {}, 14: {}}
19
20 # Remove nodes from graph
21 G1.remove_node(1) # Remove node
22 G1.remove_nodes_from([1, 11, 13, 14]) # Remove nodes
23 print(G1.nodes()) # View nodes
24 # [2, 3, 0, 6, 10, 12]

```

An edge represents a connection between two vertices. In NetworkX, an edge is represented by a tuple of the names of the corresponding vertices, e.g., `e = (node1, node2)`. Edges can have attributes such as weight and relationships. Common operations on edges include adding edges, deleting edges, defining attributes for edges, and viewing edges, and their attributes. When adding an edge to a graph in NetworkX, if the vertices of the edge do not already exist in the graph, they are automatically added to the graph.

```

1 # Add edges to the graph
2 G1.add_edge(1,5)
3 # Add an edge to G1 and automatically add vertices not in the graph
4 G1.add_edge(0,10, weight=2.7)
5 # Add an edge to G1 and set edge attributes
6 G1.add_edges_from([(1,2,{'weight':0}), (2,3,{'color':'blue'})])
7 # Add edges to the graph and set attributes
8 G1.add_edges_from([(3,6),(1,2),(6,7),(5,10),(0,1)])
9 # Add multiple edges to the graph
10 G1.add_weighted_edges_from([(1,2,3.6),[6,12,0.5]])
11 # Add multiple weighted edges to the graph: (node1,node2,weight)
12 print(G1.nodes()) # View vertices
13 # [2, 3, 0, 6, 10, 12, 1, 5, 7]
14 # Edges not in the graph are automatically added
15
16 Remove edges from the graph
17 G1.remove_edge(0,1) # Remove the edge 0-1 from the graph
18 G1.remove_edges_from([(2,3),(1,5),(6,7)])
19 # Remove multiple edges from the graph
20
21 # View edges and edge attributes
22 print(G1.edges) # View all edges
23 # [(2, 1), (3, 6), (0, 10), (6, 12), (10, 5)]

```

```

24 print(G1.get_edge_data(1,2)) # View attributes of a specific edge
25 # {'weight': 3.6}
26 print(G1[1][2]) # View attributes of a specific edge
27 # {'weight': 3.6}
28 print(G1.edges(data=True)) # View attributes of all edges
29 # [(2, 1, {'weight': 3.6}), (3, 6, {}), (0, 10, {'weight': 2.7})
30 # (6, 12, {'weight': 0.5}), (10, 5, {})]
31
32 % View information about the graph, vertices, and edges
33 \begin{lstlisting}[language=Python]
34 print(G1.nodes) # Return all vertices [node1,...]
35 # [2, 3, 0, 6, 10, 12, 1, 5, 7]
36 print(G1.edges) # Return all edges [(node1,node2),...]
37 # [(2, 1), (3, 6), (0, 10), (6, 12), (10, 5)]
38 print(G1.degree) # Return the degrees of each vertex [(node1,degree1),...]
39 # [(2, 1), (3, 1), (0, 1), (6, 2), (10, 2), (12, 1), (1, 1), (5, 1), (7, 0)]
40 print(G1.number_of_nodes()) # Return the number of vertices
41 # 9
42 print(G1.number_of_edges()) # Return the number of edges
43 # 5
44 print(G1[10])
45 # Return the attributes of all vertices adjacent to the specified vertex
46 # {0: {'weight': 2.7}, 5: {}}
47 print(G1.adj[10])
48 # Return the attributes of all vertices adjacent to the specified
49 # vertex {0: {'weight': 2.7}, 5: {}}
50 print(G1[1][2])
51 # Return the attributes of the specified edge {'weight': 3.6}
52 print(G1.adj[1][2]) # Return the attributes of the specified edge
53 # {'weight': 3.6}
54 print(G1.degree(10)) # Return the degree of the specified vertex
55 # 2
56
57 print('nx.info:', nx.info(G1))
58 # Return basic information about the graph
59 print('nx.degree:', nx.degree(G1))
60 # Return the degrees of vertices in the graph
61 print('nx.density:', nx.degree_histogram(G1))
62 # Return the distribution of degrees in the graph
63 print('nx.pagerank:', nx.pagerank(G1))
64 # Return the frequency distribution of vertices in the graph

```

A NX graph object has many properties and methods available. This table shows the commonly used ones. You can also use NetworkX to visualize the graph and analyze it.

| Function                          | Description                                               |
|-----------------------------------|-----------------------------------------------------------|
| G.has_node(n)                     | Returns True if the graph G includes node n               |
| G.has_edge(u, v)                  | Returns True if the graph G includes edge (u,v)           |
| G.number_of_nodes()               | Returns the number of nodes in graph G                    |
| G.number_of_edges()               | Returns the number of edges in graph G                    |
| G.number_of_selfloops()           | Returns the number of self-loops in graph G               |
| G.degree([nbunch, weight])        | Returns the degree of all nodes or specified nodes        |
| G.selfloop_edges([data, default]) | Returns all self-loop edges in graph G                    |
| G.subgraph([nodes])               | Extracts a subgraph from graph G1                         |
| union(G1, G2)                     | Combines graphs G and G2                                  |
| nx.info(G)                        | Returns basic information about graph G                   |
| nx.degree(G)                      | Returns the degrees of vertices in graph G                |
| nx.degree_histogram(G)            | Returns the distribution of degrees in graph G            |
| nx.pagerank(G)                    | Returns the frequency distribution of vertices in graph G |
| nx.add_star(G, [nodes], **attr)   | Adds a star network to graph G                            |
| nx.add_path(G, [nodes], **attr)   | Adds a path to graph G                                    |
| nx.add_cycle(G, [nodes], **attr)  | Adds a cycle to graph G                                   |

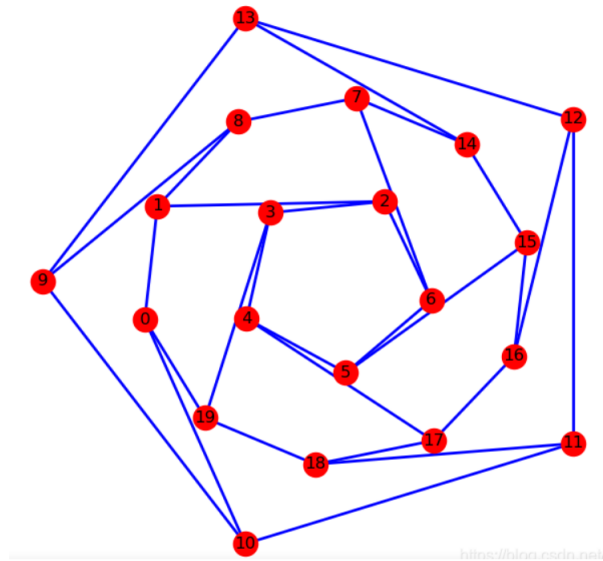


Figure 1: Visualization of Graph

## 1.4 Shorted Path Algorithm

The shortest path algorithm is a classical algorithm in the study of graph theory for computing the shortest path from one node to another in a graph. In this subsection we will discuss 2 algorithms: Dijkstra's Algorithm and A\* Algorithm.

### 1.4.1 Dijkstra's Algorithm

Dijkstra's algorithm is a popular algorithm in computer science used to find the shortest path between nodes in a graph with non-negative edge weights. It works by starting at a specific node and iteratively expanding its neighboring nodes. It maintains a priority queue of nodes with the current shortest distance from the start node, and continually selects the node with the shortest distance to expand next. This process continues until the shortest path to all reachable nodes is found. Let's consider the example. Knowing the weighted undirected

---

**Algorithm 1** Dijkstra's Algorithm

---

```
1: function DIJKSTRA(graph, start)
2:   dist  $\leftarrow \{node : \infty \text{ for } node \text{ in } graph\}$ 
3:   dist[start]  $\leftarrow 0$ 
4:   pq  $\leftarrow [(0, start)]$ 
5:   while pq is not empty do
6:     (cur_dist, cur_node)  $\leftarrow$  HeapPop(pq)
7:     if cur_dist > dist[cur_node] then
8:       continue
9:     end if
10:    for neighbor, weight in graph[cur_node].items() do
11:      new_dist  $\leftarrow$  cur_dist + weight
12:      if new_dist < dist[neighbor] then
13:        dist[neighbor]  $\leftarrow$  new_dist
14:        HeapPush(pq, (new_dist, neighbor))
15:      end if
16:    end for
17:  end while
18:  return dist
19: end function
```

---

graph as shown in figure 2, find the shortest path from node v1 to node v11. To solve this problem we should take this method step by step.

- Use function `add_weighted_edges_from()` to add wighted edges
- Define properties for each edges by tuples (node1, node2, weight)
- Use `dijkstra_path()` and `dijkstra_path_length()` to get the shortest path

```
1 import matplotlib.pyplot as plt
2 import networkx as nx
3
4 G1 = nx.Graph() # Create an empty undirected graph
5 G1.add_weighted_edges_from([(1,2,2),(1,3,8),(1,4,1),
6                             (2,3,6),(2,5,1),
7                             (3,4,7),(3,5,5),(3,6,1),(3,7,2),
```

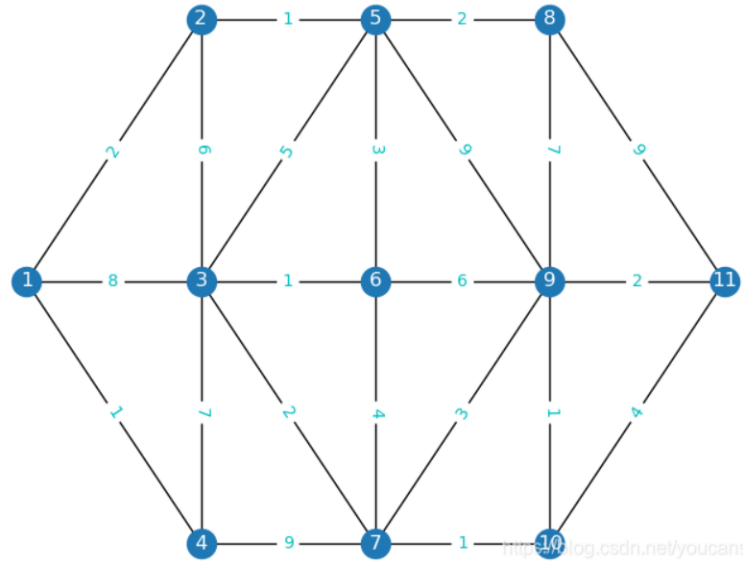


Figure 2: Weighted Undirected Graph

```

8         (4,7,9),
9         (5,6,3),(5,8,2),(5,9,9),
10        (6,7,4),(6,9,6),
11        (7,9,3),(7,10,1),
12        (8,9,7),(8,11,9),
13        (9,10,1),(9,11,2),
14        (10,11,4)])
15 # Add multiple weighted edges to the graph: (node1,node2,weight)
16 print('nx.info:',G1.nodes) # Return basic information about the graph
17
18 # Shortest weighted path between two specified vertices
19 minWPath_v1_v11 = nx.dijkstra_path(G1, source=1, target=11)
20 # Shortest weighted path from vertex 1 to vertex 11
21 print("Shortest weighted path from vertex v1 to v11: ", minWPath_v1_v11)
22 # Length of the shortest weighted path between two specified vertices
23 lMinWPath_v1_v11 = nx.dijkstra_path_length(G1, source=1, target=11)
24 # Length of the shortest weighted path
25 print("Length of the shortest weighted path from vertex v1 to v11: ", lMinWPath_v1_v11)
26
27 pos = {1: (0,4), 2: (5,7), 3: (5,4), 4: (5,1), 5: (10,7), 6: (10,4), 7: (10,1),
28        8: (15,7), 9: (15,4), 10: (15,1), 11: (20,4)}
29 # Specify the positions of vertices
30 labels = nx.get_edge_attributes(G1, 'weight')
31 # Set edge labels as 'weight'

```

```

32 nx.draw(G1, pos, with_labels=True, font_color='w') # Draw the undirected graph
33 nx.draw_networkx_edge_labels(G1, pos, edge_labels=labels, font_color='c') # Display edge w
34 plt.show()

```

The results are shown below.

```

1 nx.info: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
2 Shortest weighted path from node v1 to v11: [1, 2, 5, 6, 3, 7, 10, 9, 11]
3 Length of the shortest weighted path from node v1 to v11: 13

```

### 1.4.2 A\* Algorithm

---

#### Algorithm 2 A\* Algorithm

---

```

1: function ASTAR(start, goal)
2:   openSet  $\leftarrow$  priority queue containing start with priority 0
3:   cameFrom  $\leftarrow$  map initialized to  $\emptyset$ 
4:   gScore[start]  $\leftarrow$  0
5:   fScore[start]  $\leftarrow$  heuristic cost of start to goal
6:   while openSet is not empty do
7:     current  $\leftarrow$  node in openSet with lowest fScore
8:     if current = goal then
9:       return RECONSTRUCTPATH(cameFrom, goal)
10:    end if
11:    Remove current from openSet
12:    for each neighbor neighbor of current do
13:      tentativeGScore  $\leftarrow$  gScore[current] + cost of current to
neighbor
14:      if tentativeGScore < gScore[neighbor] then
15:        cameFrom[neighbor]  $\leftarrow$  current
16:        gScore[neighbor]  $\leftarrow$  tentativeGScore
17:        fScore[neighbor]  $\leftarrow$  gScore[neighbor] + heuristic cost of
neighbor to goal
18:        if neighbor not in openSet then
19:          Add neighbor to openSet with priority fScore[neighbor]
20:        else
21:          Update priority of neighbor in openSet to
fScore[neighbor]
22:        end if
23:      end if
24:    end for
25:  end while
26:  return failure
27: end function

```

---

A\* is an informed search algorithm that uses heuristics to guide the search towards the goal node efficiently. It is an extension of Dijkstra's algorithm with

the added ability to consider both the cost to reach a node and an estimate of the remaining cost to the destination. Here's the example code for solving previous question by A\* algorithm.

```

1  import matplotlib.pyplot as plt
2  import networkx as nx
3
4  # Define the heuristic function for A* algorithm
5  def heuristic(node, goal):
6      # Here, we use the Euclidean distance as the heuristic function
7      return ((goal[0] - node[0])**2 + (goal[1] - node[1])**2)**0.5
8
9  # Define the A* algorithm
10 def AStar(graph, start, goal):
11     openSet = {start} # Initialize the open set with the start node
12     cameFrom = {} # Initialize the map to store the parent node for each node
13     gScore = {start: 0} # Initialize the cost from start to each node
14     fScore = {start: heuristic(start, goal)} # Initialize the total cost from start to goal
15     while openSet:
16         current = min(openSet, key=lambda node: fScore.get(node, float('inf'))) # Get the node with the lowest fScore
17         if current == goal:
18             path = []
19             while current in cameFrom:
20                 path.append(current)
21                 current = cameFrom[current]
22             path.append(start)
23             return path[::-1] # Return the reconstructed path
24         openSet.remove(current)
25         for neighbor in graph[current]:
26             tentative_gScore = gScore[current] + graph[current][neighbor]['weight']
27             if tentative_gScore < gScore.get(neighbor, float('inf')):
28                 cameFrom[neighbor] = current
29                 gScore[neighbor] = tentative_gScore
30                 fScore[neighbor] = tentative_gScore + heuristic(neighbor, goal)
31                 openSet.add(neighbor)
32     return None # Failure to find a path
33
34 # Define the graph
35 G2 = nx.Graph()
36 G2.add_weighted_edges_from([(1,2,2),(1,3,8),(1,4,1),
37                             (2,3,6),(2,5,1),
38                             (3,4,7),(3,5,5),(3,6,1),(3,7,2),
39                             (4,7,9),
40                             (5,6,3),(5,8,2),(5,9,9),
41                             (6,7,4),(6,9,6),
42                             (7,9,3),(7,10,1),

```

```

43         (8,9,7),(8,11,9),
44         (9,10,1),(9,11,2),
45         (10,11,4)])
46
47     # Find the shortest path using A* algorithm
48     start_node = 1
49     goal_node = 11
50     shortest_path = AStar(G2, start_node, goal_node)
51     print("Shortest path from vertex", start_node, "to vertex", goal_node, ":", shortest_path)
52
53     # Draw the graph
54     pos = nx.spring_layout(G2)
55     nx.draw(G2, pos, with_labels=True, alpha=0.5)
56     labels = nx.get_edge_attributes(G2, 'weight')
57     nx.draw_networkx_edge_labels(G2, pos, edge_labels=labels)
58     plt.show()

```

## 2 Nonlinear Optimization

### 2.1 Linear Planning

In an optimization problem, if the objective function or constraints contain nonlinear functions, it is nonlinear planning.

Usually, nonlinear problems are much more complex and difficult than linear problems, and so is nonlinear planning. There is no unified general method or algorithm to solve nonlinear planning, and various methods have specific application scope and applicable conditions. On the other hand, many nonlinear planning problems cannot obtain the global optimal solution in practice, but only the local optimal solution or approximate optimal solution.

I will introduce some classic algorithms to solve the nonlinear problem. But remember, what's most important is not the algorithm but how to find suitable objective functions and constraints to express the problem. First, we will discuss a tool of linear planning, which is called **Simplex Method**. The simplex method is a powerful algorithm for solving linear programming problems by iteratively improving the feasible solution until an optimal solution is reached. For all algorithms in this paper, we don't care about the details too much but how we achieve them by codes. If you want to learn more, you can click on this link.

<https://zhuanlan.zhihu.com/p/388224103>

---

**Algorithm 3** Simplex Method

---

```
1: function SIMPLEX( $A, b, c$ )  $\triangleright$  Input: Constraint matrix  $A$ , right-hand side
   vector  $b$ , and objective function coefficients  $c$ 
2:   Initialize the tableau with  $A$ ,  $b$ , and  $c$ 
3:   while There exists a pivot column  $j$  with  $c_j < 0$  do  $\triangleright$  While an
   entering variable improves the objective function
4:     Choose entering variable  $x_j$  with the most negative coefficient  $c_j$ 
5:     if No such  $x_j$  exists then  $\triangleright$  Optimal solution found
6:       return Optimal solution found
7:     end if
8:     Choose pivot row  $i$  by finding the minimum ratio  $b_i/A_{ij}$  for  $A_{ij} > 0$ 
9:     if No such  $i$  exists then  $\triangleright$  Unbounded solution
10:      return Problem is unbounded
11:    end if
12:    Pivot on element  $A_{ij}$  to make it the new pivot element
13:    Update the tableau using row operations
14:  end while
15:  return Optimal solution found  $\triangleright$  Final tableau represents the optimal
   solution
16: end function
```

---

Consider the example:

$$\begin{aligned} \max \quad & f(x) = -2x_1 - 3x_2 \\ \text{s.t.} \quad & x_1 - x_2 \leq 1 \\ & -x_1 + x_2 \leq 1 \\ & x_1 + 2x_2 \leq 4 \end{aligned}$$

We can get a constraint matrix, constraint vector, and objective function coefficient vector.

$$A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & 2 \end{bmatrix} \tag{1}$$

$$b = \begin{bmatrix} 1 \\ 1 \\ 4 \end{bmatrix} \tag{2}$$

Through our analysis, we can use the simplex method to solve it.

```
1 import numpy as np
2
3 # Simplex method for solving linear programming problems
4 def simplex(A, b, c):
5     m, n = A.shape
6     # Add slack variables to convert inequality constraints to equality constraints
7     A_extended = np.hstack((A, np.eye(m)))
```

```

8     c_extended = np.concatenate((c, np.zeros(m)))
9     # Initialize tableau
10    tableau = np.vstack((np.hstack((np.zeros((1, n)), -c_extended.reshape(1, -1))),
11                          np.hstack((A_extended, b.reshape(-1, 1)))))
12    while np.any(tableau[0, :-1] < 0):
13    # While there exists a negative coefficient in the objective row
14        pivot_column = np.argmin(tableau[0, :-1])
15        # Entering variable: minimum coefficient in the objective row
16        if np.all(tableau[:, pivot_column] <= 0): # Check for unboundedness
17            return None, None # Unbounded solution
18        pivot_row = np.argmin(tableau[1:, -1] / tableau[1:, pivot_column]) + 1
19        # Exiting variable: minimum ratio
20        # Pivot operation
21        tableau[pivot_row, :] /= tableau[pivot_row, pivot_column]
22        for i in range(tableau.shape[0]):
23            if i != pivot_row:
24                tableau[i, :] -= tableau[i, pivot_column] * tableau[pivot_row, :]
25        # Extract solution and objective value
26        solution = tableau[1:, -1]
27        objective_value = -tableau[0, -1]
28        return solution, objective_value
29
30    # Example linear programming problem: maximize c^T * x subject to A * x <= b
31    A = np.array([[1, -1],
32                  [-1, 1],
33                  [1, 2]])
34    b = np.array([1, 1, 4])
35    c = np.array([-2, -3])
36
37    # Solve the linear programming problem using simplex method
38    solution, objective_value = simplex(A, b, c)
39
40    # Print the solution and objective value
41    print("Optimal solution:", solution)
42    print("Optimal objective value:", objective_value)
43
44    Optimal solution: [1. 1. 4.]
45    Optimal objective value: 0.0

```

## 2.2 Classic Algorithm

In this subsection, I will introduce 3 common algorithms: the Gradient Descent Method, the Newton's Method, and the Lagrange Multiplier Method.

### 2.2.1 Gradient Descend Method

Gradient descent is an optimization algorithm used to minimize a function by iteratively moving in the direction of the steepest descent of the function. It utilizes the information of first-order derivative to update the solution and converges to an optimal solution. We can consider an example:

---

**Algorithm 4** Gradient Descent

---

```
1: function GRADIENTDESCENT( $f, \theta, \alpha, \epsilon$ )
2:   Initialize parameters  $\theta$  randomly or with some predefined values
3:   while stopping criterion not met do
4:     Compute gradient:  $\nabla_{\theta} f(\theta)$ 
5:     Update parameters:  $\theta \leftarrow \theta - \alpha \nabla_{\theta} f(\theta)$ 
6:     Update iteration counter
7:     if convergence criteria met then
8:       return  $\theta$ 
9:     end if
10:  end while
11:  return  $\theta$ 
12: end function
```

---

$$\min_x f(x) = x^2 + 2x + 1 \quad (3)$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Define the objective function
5 def objective_function(x):
6     return x**2 + 2*x + 1 # Example function: f(x) = x^2 + 2x + 1
7
8 # Define the gradient of the objective function
9 def gradient(x):
10    return 2*x + 2 # Gradient of the example objective function: f'(x) = 2x + 2
11
12 # Gradient descent optimization
13 def gradient_descent(learning, max_iterations, tolerance):
14     x = 0 # Initial guess for the solution
15     x_vals = [x] # List to store the values of x at each
16     for i in range(max_iterations):
17         grad = gradient(x)
18         x_new = x - learning_rate * grad # Update x gradient descent
19         x_vals.append(x_new) # Record the new value of x
20         if abs(x_new - x) < tolerance:
21             break # Convergence criterion satisfied
22     x = x_new
23     return x_vals
```

```

24
25 # Parameters for gradient
26 learning_rate = 0.1 # Learning rate
27 max_iterations = 1000 # Maximum number iterations
28 tolerance = 1e-6 # Tolerance for convergence criterion
29
30 # Run gradient descent algorithm to find the optimal solution
31 opt_solution = gradient_descent(learning_rate, max_iterations, tolerance)
32
33 # Plotting the movement of solutions in the solution space
34 x_vals = np.array(opt_solution)
35 y_vals = objective_function(x_vals)
36
37 plt.figure()
38 plt.plot(x_vals, y_vals, marker='o', color='b', label='Movement of solutions')
39 plt.xlabel('x')
40 plt.ylabel('Objective function value')
41 plt.title('Movement of solutions in the solution space during gradient descent')
42 plt.legend()
43 plt.grid(True)
44 plt.show()

```

The results are shown below: Through this figure, you can intuitively under-

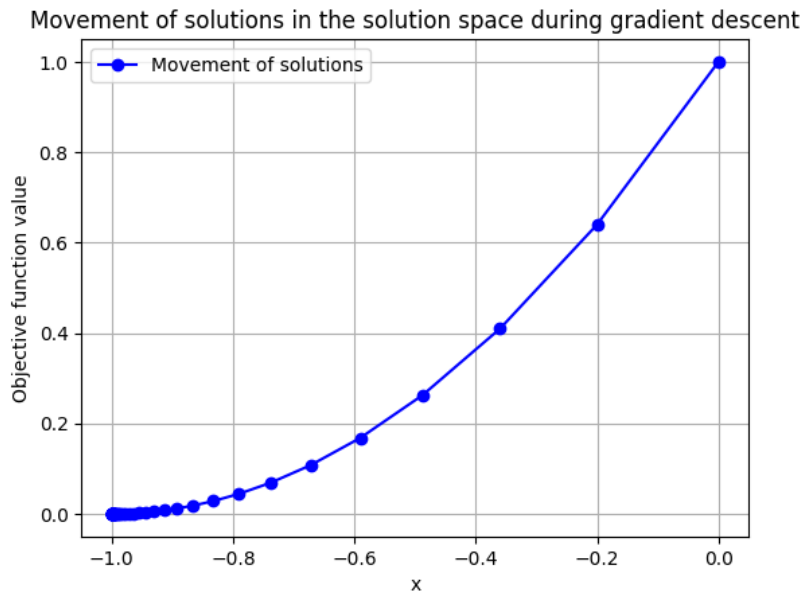


Figure 3: Trajectory of Solution for GD

stand how the solution moves to the optimal solution.

### 2.2.2 Newton's Method

Newton's method, also known as the Newton-Raphson method, is particularly useful for finding solutions to real-valued functions. Consider the same question:

---

**Algorithm 5** Newton's Method

---

```
1: function NEWTON( $f, f', x_0, \epsilon$ )
2:   Initialize initial guess  $x_0$ 
3:   while stopping criterion not met do
4:     Compute function value:  $y \leftarrow f(x)$ 
5:     Compute derivative value:  $y' \leftarrow f'(x)$ 
6:     Compute update step:  $\Delta x \leftarrow -\frac{y}{y'}$ 
7:     Update current guess:  $x \leftarrow x + \Delta x$ 
8:     if  $|\Delta x| < \epsilon$  then
9:       return  $x$  as the root
10:    end if
11:  end while
12:  return  $x$  as the root
13: end function
```

---

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Define the objective function
5 def objective_function(x):
6     return x**2 + 2*x + 1 # Example objective function:  $f(x) = x^2 + 2x + 1$ 
7
8 # Define the derivative of the objective function
9 def derivative_function(x):
10    return 2*x + 2 # Derivative of the example objective function:  $f'(x) = 2x + 2$ 
11
12 # Newton's optimization algorithm
13 def newton_method(initial_guess, max_iterations, tolerance):
14     x = initial_guess # Initial guess for the solution
15     trajectory = [x] # Store the trajectory of the solution
16     for i in range(max_iterations):
17         grad = derivative_function(x)
18         if abs(grad) < tolerance:
19             break # Convergence criterion satisfied
20         delta_x = -objective_function(x) / grad
21         # Compute the update step using Newton's method
22         x += delta_x # Update x
23         trajectory.append(x) # Store the new solution in the trajectory
```

```

24         return x, trajectory
25
26     # Parameters for Newton's method
27     initial_guess = 0 # Initial guess for the solution
28     max_iterations = 1000 # Maximum number of iterations
29     tolerance = 1e-6 # Tolerance for convergence criterion
30
31     # Run Newton's method to find the optimal solution
32     optimal_solution, trajectory = newton_method(initial_guess, max_iterations, tolerance)
33
34     # Print the optimal solution
35     print("Optimal solution:", optimal_solution)
36     print("Objective function value at optimal solution:",
37           objective_function(optimal_solution))
38
39     # Plot the trajectory of the solution in the solution space
40     trajectory = np.array(trajectory)
41     plt.plot(trajectory, objective_function(trajectory), marker='o', linestyle='-')
42     plt.xlabel('x')
43     plt.ylabel('f(x)')
44     plt.title('Trajectory of Newton\'s Method')
45     plt.grid(True)
46     plt.show()
47
48     Optimal solution: -0.9999995231628418
49     Objective function value at optimal solution: 2.2737367544323206e-13

```

## 2.3 Metaheuristic Algorithm

Nowadays, thousands of metaheuristic algorithms have already been discovered. Most of them were designed to produce papers quickly. However, we can't deny the amazing effect of some algorithms. There's no need to understand every algorithm's detailed process and mathematical proof. What should we do is call library functions. Those algorithms can be easily achieved by Mealy or scikit-opt. So I will just simply introduce 2 common algorithms and show their effect on the previous problem.

### 2.3.1 Genetic Algorithm(GA)

Genetic Algorithm (GA) is a population-based optimization algorithm inspired by the process of natural selection and genetics. It is often used to find approximate solutions to optimization and search problems. Here's how it works:

- Initialization: GA starts by initializing a population of potential solutions randomly or with some heuristic.

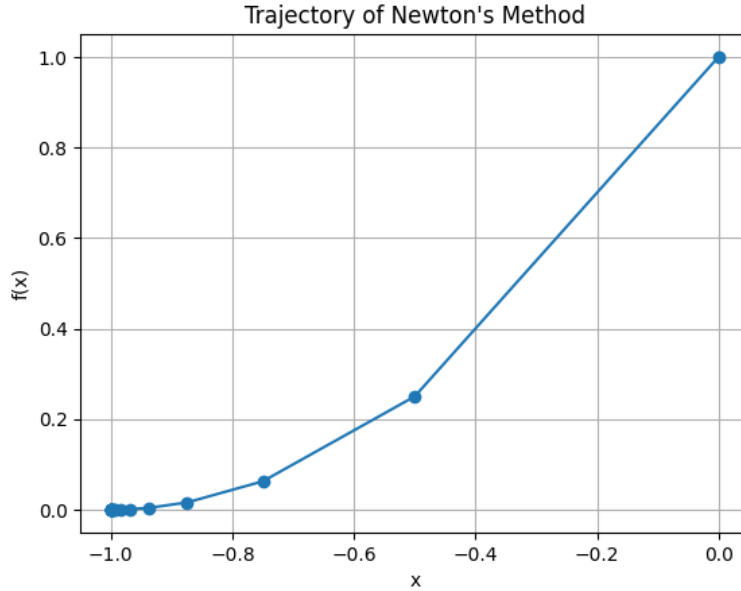


Figure 4: Trajectory of Solution for Newton's Method

- **Evaluation:** Each individual in the population is evaluated using a fitness function, which measures how good each solution is with respect to the optimization problem.
- **Selection:** Individuals are selected from the population based on their fitness scores. Individuals with higher fitness scores have a higher chance of being selected for reproduction.
- **Crossover:** Selected individuals are paired together, and crossover (also known as recombination) is applied to produce new offspring. This mimics the process of mating in natural evolution.
- **Mutation:** Some individuals undergo mutation, where random changes are introduced into their genetic material. Mutation helps introduce diversity into the population and prevent premature convergence to suboptimal solutions.
- **Replacement:** The new offspring and mutated individuals replace the least fit individuals in the population, ensuring that the population size remains constant.
- **Termination:** The process is repeated for a certain number of generations or until a termination criterion is met, such as finding a solution of acceptable quality or reaching a maximum number of iterations.

```

1  import numpy as np
2
3
4  def schaffer(p):
5      '''
6      This function has plenty of local minimum, with strong shocks
7      global minimum at (0,0) with value 0
8      '''
9      x1, x2 = p
10     x = np.square(x1) + np.square(x2)
11     return 0.5 + (np.square(np.sin(x)) - 0.5) / np.square(1 + 0.001 * x)
12
13 from sko.GA import GA
14
15 ga = GA(func=schaffer, n_dim=2, size_pop=50, max_iter=800,
16         prob_mut=0.001, lb=[-1, -1], ub=[1, 1], precision=1e-7)
17 best_x, best_y = ga.run()
18 print('best_x:', best_x, '\n', 'best_y:', best_y)
19
20 # Plot the convergence curve
21 plt.plot(ga.generation_best_Y)
22 plt.xlabel('Iteration')
23 plt.ylabel('Objective Function Value')
24 plt.title('Convergence Curve of Genetic Algorithm')
25 plt.grid(True)
26 plt.show()
27
28 best_x: [-8.94069698e-08 -2.98023233e-08]
29 best_y: [0.]

```

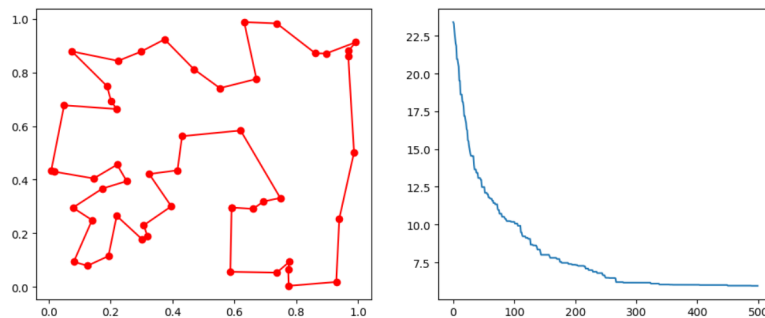


Figure 5: Convergence Curve of GA

### 2.3.2 Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is a population-based optimization algorithm inspired by the social behavior of birds flocking or fish schooling. It is used to find optimal solutions by iteratively improving a swarm of candidate solutions. Here's how it works:

- Initialization: PSO starts by initializing a population of particles, each representing a potential solution to the optimization problem. Each particle has a position and a velocity.
- Evaluation: The fitness of each particle is evaluated using a fitness function, which measures how good each solution is with respect to the optimization problem.
- Velocity Update: Each particle adjusts its velocity based on its current velocity, its best previous position (personal best), and the best position found by any particle in the swarm (global best).
- Position Update: Each particle updates its position by moving according to its current velocity. This movement is guided by its own experience and the collective experience of the swarm.
- Best Position Update: Each particle updates its personal best position if its current position has a better fitness value than its previous best position.
- Global Best Update: The global best position found by any particle in the swarm is updated if a particle finds a new best position.
- Termination: The process is repeated for a certain number of iterations or until a termination criterion is met, such as finding a solution of acceptable quality or reaching a maximum number of iterations.

```
1 def demo_func(x):
2     x1, x2, x3 = x
3     return x1 ** 2 + (x2 - 0.05) ** 2 + x3 ** 2
4
5 from sko.PSO import PSO
6
7 pso = PSO(func=demo_func, n_dim=3, pop=40, max_iter=150, lb=[0
8     , -1, 0.5], ub=[1, 1, 1], w=0.8, c1=0.5, c2=0.5)
9 pso.run()
10 print('best_x is ', pso.gbest_x, 'best_y is', pso.gbest_y)
11
12 plt.plot(pso.gbest_y_hist)
13 plt.xlabel('Iteration')
14 plt.ylabel('Objective Function Value')
15 plt.title('Convergence Curve of Genetic Algorithm')
```

```
16 plt.grid(True)
17 plt.show()
```

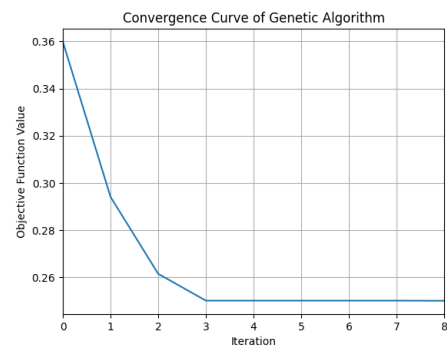


Figure 6: Convergence Curve of PSO